

SQL Cheat Sheet: Accessing Databases using Python

SQLite

Topic	Syntax	Description	Example
connect()	<code>sqlite3.connect()</code>	Create a new database and open a database connection to allow <code>sqlite3</code> to work with it. Call <code>sqlite3.connect()</code> to create a connection to the database <code>INSTRUCTOR.db</code> in the current working directory, implicitly creating it if it does not exist.	<pre>import sqlite3 con = sqlite3.connect("INSTRUCTOR.db")</pre>
cursor()	<code>con.cursor()</code>	To execute SQL statements and fetch results from SQL queries, use a database cursor. Call <code>con.cursor()</code> to create the Cursor.	<pre>cursor_obj = con.cursor()</pre>
execute()	<code>cursor_obj.execute()</code>	The <code>execute</code> method in Python's <code>SQLite</code> library allows to perform SQL commands, including retrieving data from a table using a query like "Select * from <code>table_name</code> ." When you execute this command, the result is obtained as a collection of table data stored in an object, typically in the form of a list of lists.	<pre>cursor_obj.execute('insert into INSTRUCTOR values (1, 'Rav</pre>
fetchall()	<code>cursor_obj.fetchall()</code>	The <code>fetchall()</code> method in Python retrieves all the rows from the result set of a query and presents them as a list of tuples.	<pre>statement = '''SELECT * FROM INSTRUCTOR''' cursor_obj.execute(statement) output_all = cursor_obj.fetchall() for row_all in output_all: print(row_all)</pre>
fetchmany()	<code>cursor_obj.fetchmany()</code>	The <code>fetchmany()</code> method retrieves the subsequent group of rows from the result set of a query rather than just a single row. To fetch a few rows from the table, use <code>fetchmany(numberofrows)</code> and mention how many rows you want to fetch.	<pre>statement = '''SELECT * FROM INSTRUCTOR''' cursor_obj.execute(statement) output_many = cursor_obj.fetchmany(2) for row_many in output_many: print(row_many)</pre>
read_sql_query()	<code>read_sql_query()</code>	<code>read_sql_query()</code> is a function provided by the <code>Pandas</code> library in Python, and it is not specific to <code>MySQL</code> . It is a generic function used for executing SQL queries on various database systems, including <code>MySQL</code> , and retrieving the results as a <code>Pandas DataFrame</code> .	<pre>df = pd.read_sql_query("select * from instructor;", conn)</pre>

shape	dataframe.shape	It provides a tuple indicating the shape of a DataFrame or Series, represented as (number of rows, number of columns).	df.shape
close()	con.close()	con.close() is a method used to close the connection to a MySQL database. When called, it terminates the connection, releasing any associated resources and ensuring the connection is no longer active. This is important for managing database connections efficiently and preventing resource leaks in your MySQL database interactions.	con.close()
CREATE TABLE	CREATE TABLE table_name (column1 datatype constraints, column2 datatype constraints, ...);	The CREATE TABLE statement is used to define and create a new table within a database. It specifies the table's name, the structure of its columns (including data types and constraints), and any additional properties such as indexes. This statement essentially sets up the blueprint for organizing and storing data in a structured format within the database.	CREATE TABLE INTERNATIONAL_STUDENT_TEST_SCORES (country VARCHAR(50), first_name VARCHAR(50), last_name VARCHAR(50), test_score INT);
barplot()	seaborn.barplot(x="x-axis_variable", y="y-axis_variable", data=data)	seaborn.barplot() is a function in the Seaborn Python data visualization library used to create a bar plot, also known as a bar chart. It is particularly used to display the relationship between a categorical variable and a numeric variable by showing the average value for each category.	import seaborn seaborn.barplot(x='Test_Score',y='Frequency', data=dataframe)
read_csv()	df = pd.read_csv('file_path.csv')	read_csv() is a function in Python's Pandas library used for reading data from a Comma-Separated Values (CSV) file and loading it into a Pandas DataFrame. It's a common method for working with tabular data stored in CSV format	import pandas df = pandas.read_csv('https://data.cityofchicago.org/resourc
to_sql()	df.to_sql('table_name', index=False)	df.to_sql() is a method in Pandas, a Python data manipulation library used to write the contents of a DataFrame to a SQL database. It allows to take data from a DataFrame and store it structurally within a SQL database table.	import pandas df = pandas.read_csv('https://data.cityofchicago.org/resourc df.to_sql("chicago_socioeconomic_data", con, if_exists='repl
read_sql()	df = pd.read_sql(sql_query, conn)	read_sql() is a function provided by the Pandas library in Python for executing SQL queries and retrieving the results into a DataFrame from an SQL database. It's a convenient way to	selectQuery = "select * from INSTRUCTOR" df = pandas.read_sql(selectQuery, conn)

integrate SQL database interactions into your data analysis workflows.

Db2

Topic	Syntax	Description	Example
connect()	<pre>conn = ibm_db.connect('DATABASE=dbname; HOST=hostname;PORT=port;UID=username; PWD=password;', '', '')</pre>	ibm_db.connect() is a Python function provided by the ibm_db library, which is used for establishing a connection to an IBM Db2 or IBM Db2 Warehouse database. It's commonly used in applications that need to interact with IBM Db2 databases from Python.	<pre>import ibm_db conn = ibm_db.connect('DATABASE=mydb; HOST=example.com;PORT=50000;UID=myuser; PWD=mypassword;', '', '')</pre>
server_info()	<pre>ibm_db.server_info()</pre>	ibm_db.server_info(conn) is a Python function provided by the ibm_db library, which is used to retrieve information about the IBM Db2 server to which you are connected.	<pre>server = ibm_db.server_info(conn) print ("DBMS_NAME: ", server.DBMS_NAME) print ("DBMS_VER: ", server.DBMS_VER) print ("DB_NAME: ", server.DB_NAME)</pre>
close()	<pre>con.close()</pre>	con.close() is a method used to close the connection to a db2 database. When called, it terminates the connection, releasing any associated resources and ensuring the connection is no longer active. This is important for managing database connections efficiently and preventing resource leaks in your db2 database interactions.	<pre>con.close()</pre>
exec_immediate()	<pre>sql_statement = "SQL statement goes here" stmt = ibm_db.exec_immediate(conn, sql_statement)</pre>	ibm_db.exec_immediate() is a Python function provided by the ibm_db library, which is used to execute an SQL statement immediately without the need to prepare or bind it. It's commonly used for executing SQL statements that don't require input parameters or don't need to be prepared in advance.	<pre># Lets first drop the table INSTRUCTOR in case i dropQuery = "drop table INSTRUCTOR" dropStmt = ibm_db.exec_immediate(conn, dropQuery</pre>

Author(s)

[Abhishek Gagneja](#)

[D.M Naidu](#)



Skills Network